

```

1  (*
2                                     CS51 Lab 5
3                               Variants, algebraic types, and pattern matching
4  *)
5
6  (*=====
7  Readings:
8
9      This lab builds on material from Chapters 11-11.2 of the textbook
10     <http://book.cs51.io>, which should be read before the lab session.
11
12 Objective:
13
14     In this lab you'll get practice with built-in algebraic data types,
15     including product types (like tuples and records) and sum types
16     (like variants), and the expressive power that arises from combining
17     both. A theme will be the requirement of and checking for
18     consistency with invariants.
19
20 NOTE:
21
22     Since we ask that you define types in this lab, you must complete
23     certain exercises before this will compile with the testing
24     framework on the course grading server. Exercises 1, 2, 6, and 10
25     are required for full compilation. If you want to "peek" at the
26     right type definitions, you can check out
27     <http://url.cs51.io/lab5-1>.
28     =====*)
29
30  (*=====
31  Part 1: Colors as an algebraic data type
32
33  In this lab you'll use algebraic data types to create several data
34  structures.
35
36  Ultimately, you'll define several types and structures that allow you to
37  create a family tree. To do this, you need to create a type to store a
38  set of biographical information about a person, like name, birthdate,
39  and favorite color. This set of data is different from the enrollment
40  data from the prior lab, so you'll need to create a new type.
41
42  You might be tempted to do something simple like
43
44      type person = { name : string; favorite : string; birthday : string } ;;
45
46  Let's consider why this may not be appropriate by evaluating the type
47  for each record field individually.
48
49  It seems reasonable for a name to be a string (though personal name
50  structures can be quite complex; see
51  <https://en.wikipedia.org/wiki/Personal\_name>), so let's declare that
52  complete and move on.

```

```

53
54 The "favorite" field is more problematic. Although we named it such for
55 simplicity, it doesn't convey very well that we intended for this field
56 to represent a person's favorite *color*. This could be resolved with
57 some documentation, but is not enforced at any level other than hope.
58 Further, it's very likely that many people would select one of a subset
59 of simple colors. Let's fix this issue first.
60
61 .....
62 Exercise 1: Define a new algebraic type, called `color_label`, whose
63 values can be any of red, orange, yellow, green, blue, indigo, or
64 violet, each specified by an appropriate constructor.
65 .....*)
66
67 type color_label = NotImplemented ;;
68
69 (* This is a good start, but doesn't allow for definition of *all* of
70 the colors that, say, a computer display might be able to present. Let's
71 make it more expressive.
72
73 One of the most commonly used methods of representing color in digital
74 devices is as an "RGB" value: a triplet of values to represent red,
75 green, and blue components that, through additive mixing, produce the
76 wide array of colors our devices render.
77
78 Commonly, each of the red, green, and blue values are made up of a
79 single 8-bit (1-byte) integer. Since one byte represents  $2^{**} 8 = 256$ 
80 discrete values, there are over 16.7 million ( $256 * 256 * 256$ )
81 possible colors that can be represented with this method.
82
83 The three components that make up an RGB color are referred to as
84 "channels". In this 8-bit-per-channel model, a value of 0 represents no
85 color and a value of 255 represents the full intensity of that color.
86 Some examples:
87
88     R | G | B | Color
89     ---|---|---|-----
90     255|  0|  0| Red
91     164| 16| 52| Crimson
92     255|165|  0| Orange
93     255|255|  0| Yellow
94     0 | 64|  0| Dark green
95     0 |255|  0| Green
96     0 |255|255| Cyan
97     0 |  0|255| Blue
98     75|  0|130| Indigo
99     240|130|240| Violet
100
101 .....
102 Exercise 2: Define an algebraic color type that supports either
103 `Simple` colors (from the `color_label` type you defined previously)
104 or `RGB` colors, which would incorporate a tuple of values for the
105 three color channels. You'll want to use `Simple` and `RGB` as the
106 value constructors in this new variant type.

```

```

107 .....*)
108
109 type color = NotImplemented ;;
110
111 (* There is an important assumption about the RGB values that determine
112 whether a color is valid or not. The RGB type presupposes an
113 *invariant*, that is, a condition that we assume to be true in order for
114 the type to be valid:
115
116     INVARIANT: The red, green, and blue channels must each be a
117     non-negative 8-bit int. Therefore, each channel must be in the range
118     [0, 255].
119
120 Since OCaml, unlike some other languages, does not have native support
121 for unsigned 8-bit integers, you should ensure the invariant remains
122 true in your code. (You might think to use the OCaml `char` type --
123 which is an 8-bit character -- but this would be an abuse of the type.
124 In any case, thinking about invariants will be useful practice for
125 upcoming problem sets.)
126
127 We'll want a function to validate the invariant for RGB color values.
128 (Simple colors are assumed always valid.) There are several approaches
129 to building such functions, which differ in their types, and for which
130 we'll use different naming conventions:
131
132 * valid_rgb : color -> bool
133
134     Returns true if the color argument is valid, false otherwise.
135
136 * validated_rgb : color -> color
137
138     Returns its argument unchanged if it is a valid color, and raises an
139     appropriate exception otherwise.
140
141 * validate_rgb : color -> unit
142
143     Returns unit; raises an appropriate exception if its argument is not
144     a valid color.
145
146 The name prefixes "valid_", "validated_", and "validate_" are intended
147 to be indicative of the different approaches to validation.
148
149 In this lab, we'll use the "validated_" approach and naming
150 convention, though you may want to think about the alternatives. (In
151 the next lab, we use the "valid_" alternative approach.)
152
153 .....
154 Exercise 3: Write a function `validated_rgb` that accepts a `color` and
155 returns that color unchanged if it's valid. However, if its argument is
156 not a valid color (that is, the invariant is violated), it raises an
157 `Invalid_color` exception (defined below) with a useful message.
158
159 For instance:
160

```

```

161     # validated_rgb (RGB (10, 20, 30)) ;;
162     - : color = RGB (10, 20, 30)
163     # validated_rgb (RGB (10, 20, 300)) ;;
164     Exception: Invalid_color "bad blue channel".
165     # validated_rgb (Simple Red) ;;
166     - : color = Simple Red
167
168     .....*)
169
170 exception Invalid_color of string ;;
171
172 let validated_rgb =
173     fun _ -> failwith "validated_rgb not implemented" ;;
174
175 (*.....*)
176 Exercise 4: Write a function `make_color` that accepts three integers
177 for the channel values and returns a value of the color type. Be sure
178 to verify the invariant.
179 .....*)
180
181 let make_color =
182     fun _ -> failwith "make_color not implemented" ;;
183
184 (*.....*)
185 Exercise 5: Write a function `rgb_of_color` that accepts a `color` and
186 returns a 3-tuple of integers representing that color. This is trivial
187 for `RGB` colors, but not quite so easy for the hard-coded `Simple`
188 colors. Fortunately, we've already provided RGB values for simple
189 colors in the table above.
190 .....*)
191
192 let rgb_of_color =
193     fun _ -> failwith "rgb_of_color not implemented" ;;
194
195 (*=====
196 Part 2: Dates as a record type
197
198 Now let's move on to the last data type that will be used in the
199 biographical data type: the date field.
200
201 Above, we naively proposed a string for the date field. Does this make
202 sense for this field? Arguably not, since it will make comparison and
203 calculation extremely difficult.
204
205 Dates are frequently needed in programming, and OCaml (like many
206 languages) supports them through a library module; the `Unix` module
207 provides a `tm` data type for dates and times. Normally, we would
208 reduce duplication of code by relying on that module (the edict of
209 irredundancy), but for the sake of practice you'll develop your
210 own simple version.
211
212 .....*)
213 Exercise 6: Create a type `date` that supports values for years,
214 months, and days. First, consider what types of data each value should

```

```

215 be. Then, consider the implications of representing the overall data
216 type as a tuple or a record.
217 .....*)
218
219 type date = NotImplemented ;;
220
221 (* After you've thought it through, go to <http://url.cs51.io/lab5-1> to
222 see how we implemented the `date` type. If you picked differently, why
223 did you choose that way? Why might our approach be preferable?
224
225 .....
226 Exercise 7: Change your `date` data type, above, to implement it in a
227 manner identical to our method. If no changes are required...well, that
228 was easy.
229 .....
230
231 Like the color type, above, date values obey invariants. In fact, the
232 invariants for this type are more complex: we must ensure that days
233 fall within an allowable range depending on the month, and even on the
234 year.
235
236 The invariants are as follows:
237
238 - For our purposes, we'll only support non-negative years.
239
240 - January, March, May, July, August, October, and December have 31
241   days.
242
243 - April, June, September, and November have 30 days.
244
245 - February has 28 days in common years, 29 days in leap years.
246
247 - Leap years are years that can be divided by 4, but not by 100,
248   unless by 400.
249
250 You may find Wikipedia's leap year algorithm pseudocode useful:
251 <https://en.wikipedia.org/wiki/Leap_year#Algorithm>
252
253 .....
254 Exercise 8: Create a `validated_date` function that raises
255 `Invalid_date` if the invariant is violated, and returns the date
256 unchanged if valid.
257 .....*)
258
259 exception Invalid_date of string ;;
260
261 let validated_date =
262   fun _ -> failwith "validated_date not implemented" ;;
263
264 (*.....
265 Exercise 9: Define a function `string_of_date` that returns a string
266 representing its date argument. For example,
267
268   # string_of_date {year = 1706; month = 1; day = 17} ;;

```

```

269     - : string = "January 17, 1706"
270     .....*)
271
272 let string_of_date =
273     fun _ -> failwith "string_of_date not implemented" ;;
274
275 (*=====
276 Part 3: Persons as an algebraic data type
277
278 Now, combine all of these different types to define a person record,
279 with a name, a favorite color, and a birthdate.
280
281 .....
282 Exercise 10: Define a `person` record type. Use the field names
283 `name`, `favorite`, and `birthdate`.
284 .....*)
285
286 type person = NotImplemented ;;
287
288 (* Just for fun, here's a function to open up a graphics window and
289 display a person's information. Try it out! *)
290
291 (* display_person person -- Displays `person`'s name and birth date in
292    a graphics window on a background of their favorite color. *)
293
294 let display_person ({name; favorite; birthdate} : person) : unit =
295     let open Graphics in
296     open_graph "";
297     resize_window 200 60;
298     let r, g, b = rgb_of_color favorite in
299     set_color (rgb r g b);
300     fill_rect 0 0 200 60;
301     set_color white;
302     moveto 20 40;
303     draw_string name;
304     moveto 20 20;
305     draw_string (string_of_date birthdate);
306     ignore (read_key ()) ;;
307
308 (*
309 let () = display_person {name = "Ben Franklin";
310                          favorite = Simple Blue;
311                          birthdate = {year = 1706; month = 1; day = 17}} ;;
312 *)

```